

RAG Framework 비교

Overview

RAG Framework 종합 비교

- 실무에서는 특정 프레임워크 하나만 고집하지 않는다.
- 인덱싱은 Llamaindex, 복잡한 에이전트 흐름은 LangGraph, 프롬프트 자동 튜닝은 DSPy를 혼용하는 경우가 많다.

프레임워크	특징	개발편의	학습곡선
LangChain	범용 오케스트레이션	중	중
LlamIndex	인덱싱/검색 특화	상	하~중
Haystack	컴포넌트 파이프라인	중	중
RAWFlow	End-to-End 엔진	상(웹 UI)	하
DSPy	프롬프트 최적화	하	상

LlamaIndex

☑ LlamaIndex 의 정체성

- 데이터를 LLM이 쓰기 좋게 인덱싱·질의'에 특화된 프레임워크

☑ 강점·약점 + 언제 LlamaIndex인가

- 빠른 데이터 적재와 다양한 인덱싱 구조가 강점, 세밀한 내부 제어가 어렵고 복잡한 에이전트 구축에 한계가 있음
- 검색·인덱싱은 LlamaIndex, 복잡한 에이전트 오케스트레이션은 LangGraph로 병용하기도 한다.

항목	상세
정체성	데이터를 LLM이 활용하기 좋게 인덱싱·질의하는 데 특화된 '문서 중심 RAG의 가장 빠른 길
특징	질문 유형에 따라 VectorStoreIndex(의미 검색) 또는 SummaryIndex(문서 요약) 등 적합한 인덱스 모드를 선택해야함
강점	from_documents 한 줄로 청킹, 임베딩, 적재가 완료되어 빠른 프로토타이핑에 최적
약점	내부 제어 덜 투명, 복잡한 에이전트는 LangGraph 우세
언제	문서 중심 RAG·빠른 프로토타입·요약/계층 질의가 필요할 때 1순위

Haystack

☑ Haystack 의 정체성

- 각 단계를 컴포넌트로 만들고 포트를 명시적으로 연결하여, YAML 직렬화 및 배포 운영에 친화적인 구조를 제공

☑ 강점.약점 + 언제 Haystack인가

- 명시적 흐름과 서빙 안정성이 뛰어나지만 포트 연결 코드가 길어지며, 동적 에이전트 구성은 제약이 있다.
- 보일러플레이트(Boilerplate) : 최소한의 수정만 거쳐 여러 곳에 반복적으로 사용되는 표준화된 텍스트나 코드

항목	상세
정체성	파이프라인의 각 단계를 입출력 포트가 명확히 정의된 독립적인 컴포넌트로 다루는 프레임워크
특징	명시적 포트연결, YAML 배포, 하이브리드&리랭킹지원
강점	명시적 파이프라인(가시성 ↑).운영/서빙 성숙.컴포넌트 교체 용이
약점	포트 연결 보일러플레이트 많음, 동적 에이전트는 LangGraph보다 약함
언제	검색 중심 서비스를 명시적으로 관리.서빙.협업해야 할 때

RAGFlow

☑ RAGFlow 의 정체성

- 코드로 조립하는 프레임워크가 아닌, Docker로 기동하여 웹 UI와 API로 사용하는 완제품 형태의 엔드투엔드 엔진

☑ 강점·약점 + 다른 F/W 와의 차이

- 비개발자도 정교한 문서 기반 시스템을 구축할 수 있으나, 리소스 요구량이 크고 코드 레벨의 커스텀 제약이 존재

항목	상세
정체성	띄워서 쓰는 완제품 RAG - 비개발 팀·복잡 문서에 강점, 코드 세밀 제어는 제한.
특징	레이아웃, 표, 스캔 OCR을 정교하게 다루는 '깊은 문서 이해' 능력과 템플릿 기반 청킹, 인용 시각화를 제공
강점	문서 파싱 품질(표·스캔)·웹UI·청킹 템플릿·인용 - 비개발자도 구축
약점	무겁다(Docker·리소스)·코드 세밀 제어 제한·커스텀 유연성 ↓
차이	다른 넷은 '코드로 조립하는 프레임워크', RAGFlow는 '띄워서 쓰는 엔진'

✓ DSPy의 정체성

- 프롬프트를 직접 작성하지 않고, 입력/출력 구조(Signature)와 모듈을 정의하여 데이터로 프롬프트를 컴파일한다.

✓ 강점·약점 + 언제 DSPy 인가

- 컴파일 단계에서 LLM 다회 호출로 비용이 들지만 오프라인 1회 수행 후 결과물을 재사용하며, 데이터 기반 평가 기준이 명확한 시스템 구축에 적합

항목	상세
정체성	프롬프트를 '작성물'이 아니라 '학습 결과물'로 본다
특징	프롬프트를 직접 작성하지 않고, 모듈로 정의한 뒤 데이터 기반으로 프롬프트를 자동 생성 및 최적화 한다.
강점	모델이 변경되거나 요구사항이 바뀌어도 재컴파일 한 번으로 품질을 유지할 수 있다.
약점	최적화 단계에서 LLM API 호출이 다량 발생하여 시간과 비용이 소모 개발자의 초기 학습 곡선이 높다
언제	품질을 데이터로 끌어올리고 평가 기준이 명확한 실험·연구·고품질 RAG

RAG Framework 종합 비교

☑️ 비교전제 – 우열이 아니라 결

- 목적·추상화·운영 방식이 달라 '무엇이 최고'는 성립하지 않는다.
- LlamaIndex=인덱싱, Haystack=파이프라인, RAGFlow=문서이해, DSPy=프롬프트최적화, LangChain=오케스트레이션

☑️ 선택가이드 – 시나리오별 추천

- 빠른 프로토타이핑, 명시적 서빙, 복잡 문서, 데이터 최적화 등 상황에 맞춘 전략적 선택이 필요하다.

상황/요구	1순위	이유
문서 넣고 바로 질의·다양한 인덱스	LlamaIndex	한 줄 인덱싱·인덱스 다양성
흐름 명시 관리·서빙·검색 제품	Haystack	파이프라인 가시성·YAML 서빙
복잡 문서(스캔·표)·비개발 운영	RAGFlow	깊은 문서 이해·웹UI
복합 분기·에이전트·툴 결합	LangChain(+LangGraph)	오케스트레이션·생태계
품질을 데이터로 최적화	DSPy	프롬프트 자동 튜닝·재현성